Topic #:-

Non-Comparison Sorting algorithm:

=> Algorithms that perform sorting without comparing the elements called by making certain assumptions about the data.

They are called non-comparison sorting.

Examples:

1) Counting Sort (indexes using key values)
2) Radix Sort (examines individual bits of k
3) Bucket Sort (examines bits of keys)

=> These are linear sorting algorithms.

=> Linear Sorts are not "comparison sorts".

=> These algorithms do not need to go through the comparison decision tree.

=> Consequently, the $\Omega(n \log n)$ lower bound does not apply to them.

Limitations of Comparison Sorting:

=> These are fundamental limits on the performance of comparison sorts.

=> To sort n elements, comparison sort must make $\Omega(n \lg n)$ comparisons in the worst case.

That is a comparison sort must have a lower bound of $\Omega(n \lg n)$ comparison operations, which is known as linear or linearithmic time.

Q# Write the name of asymptotically optimal comparison sort?

Heapsort
mergesort

Q#: What is the smallest possible depth of a leaf in a decision tree for a comparison sort?

∴ n−1

=> because to find the minimum (or maximum) of n elements requires n−1 comparisons.

=> Hence, the smallest possible depth is n−1.

Counting Sort

ounting sort, sorts the elements of
array by counting the number
occurrences of each unique element
in the array.
Counting sort assumes that each
the n input elements is an integer
the range 0 to k.
is the number of elements
is the highest value element.
Counting sort determines for each
nput element x, the number of
ements less than x.
And it uses this information to
ace element x directly into its
osition in the output array.
The algorithm uses three arrays:
nput Array : A[1...n]
utput array : B[1...n]
Temporary Array : C[1...k]

able: Yes
-place: No, out-of-place

Time Complexity:

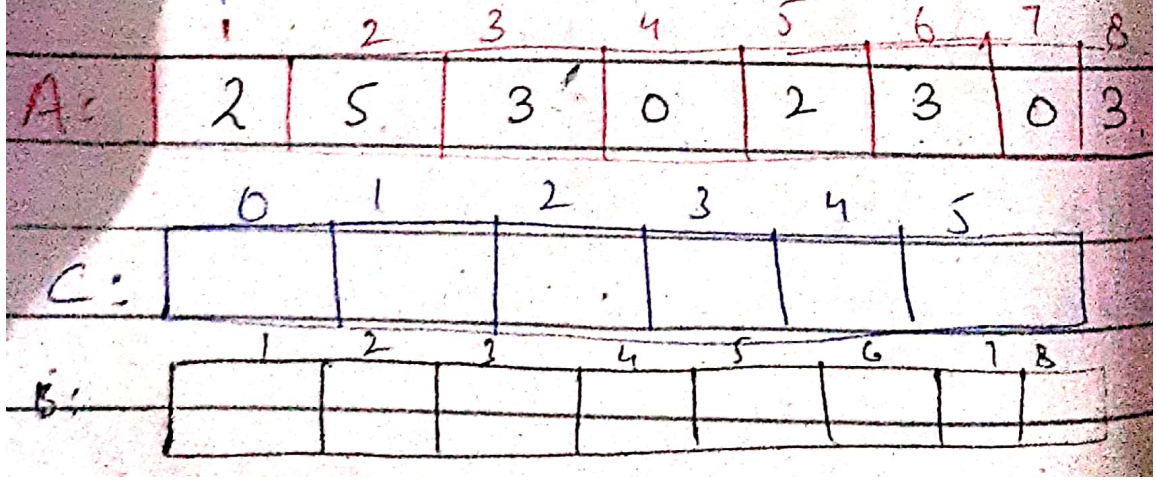
=> $O(n+k)$

Note: Suitable only for small range of values.

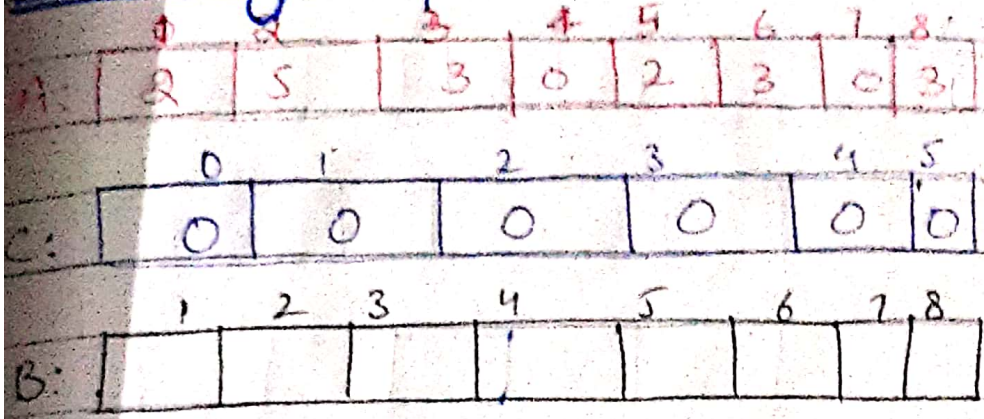
Algorithm:

```
1- Counting_Sort(A, B, k)
2- Let C[0...k] be a new array
3- for i=0 to k —— Θ(k)
       C[i]=0 —— k
4- for j=1 to A.length or n —— Θ(n)
       C[A[j]] = C[A[j]] + 1 —— n
6- for j=n or A.length down to 1 —— Θ(n)
       B[C[A[j]]] = A[j] —— n
       C[A[j]] = C[A[j]] - 1
5- for i=1 to k —— Θ(k)
       C[i] = C[i] + C[i-1] —— k
```

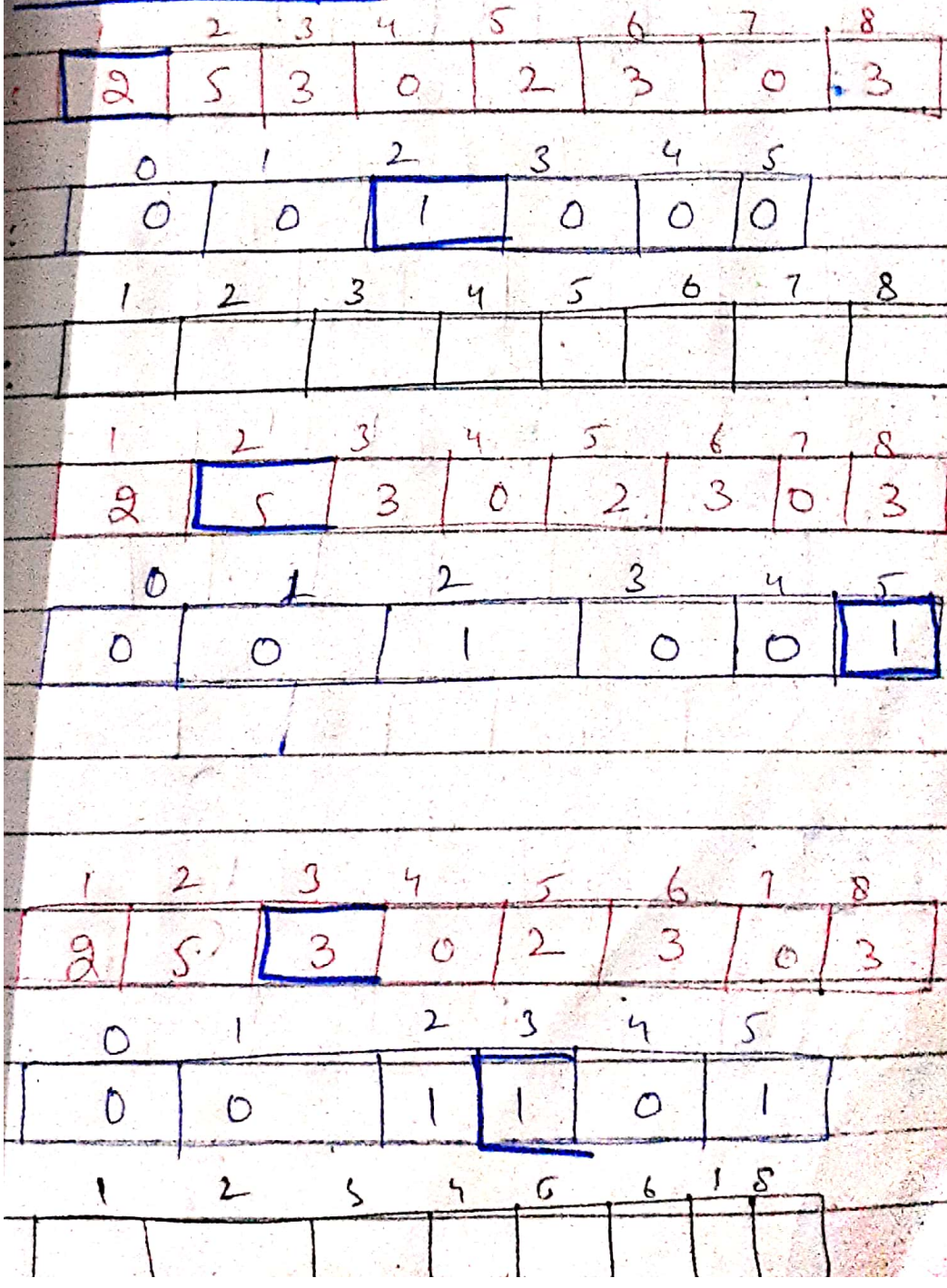
Example:

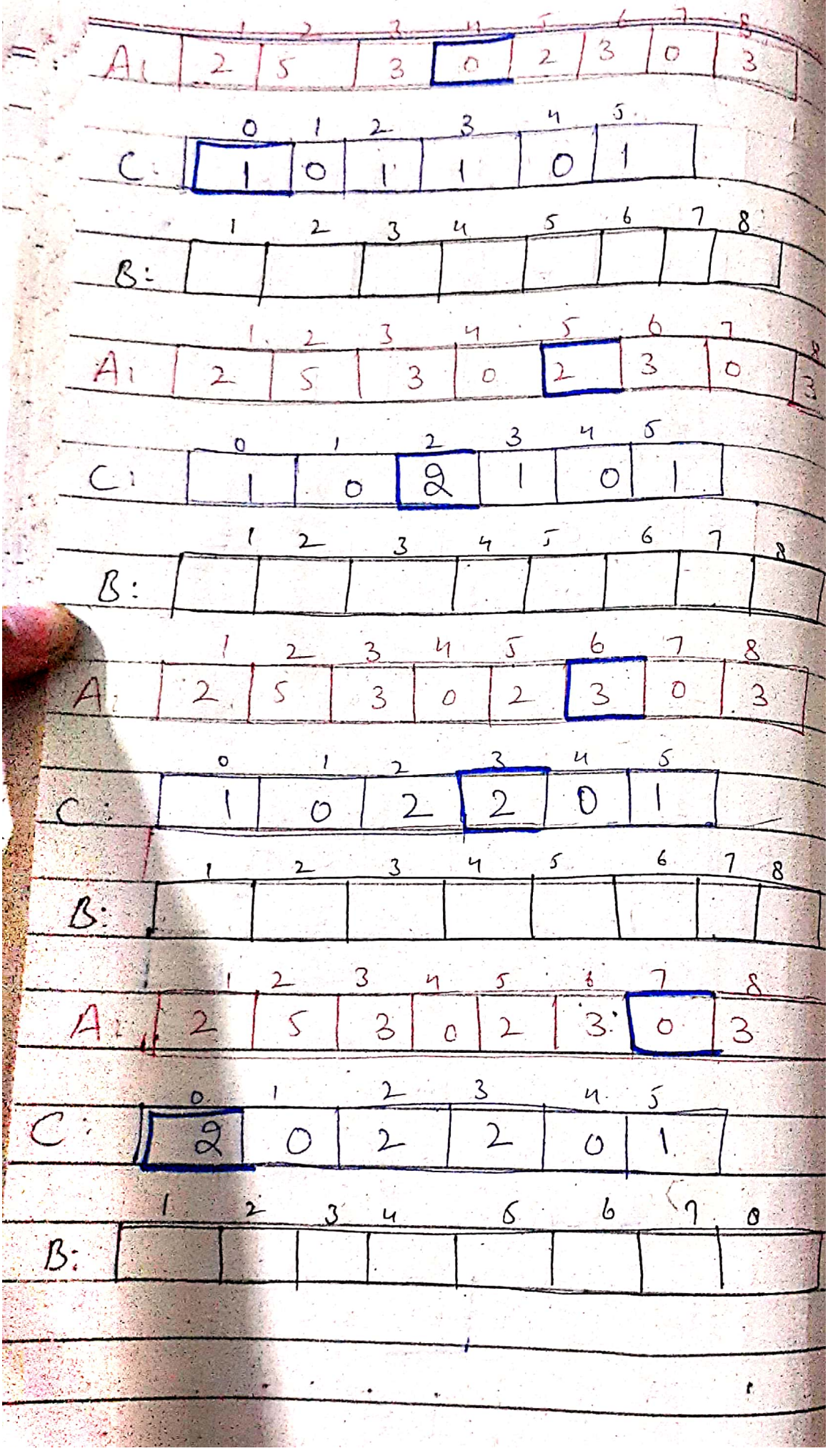


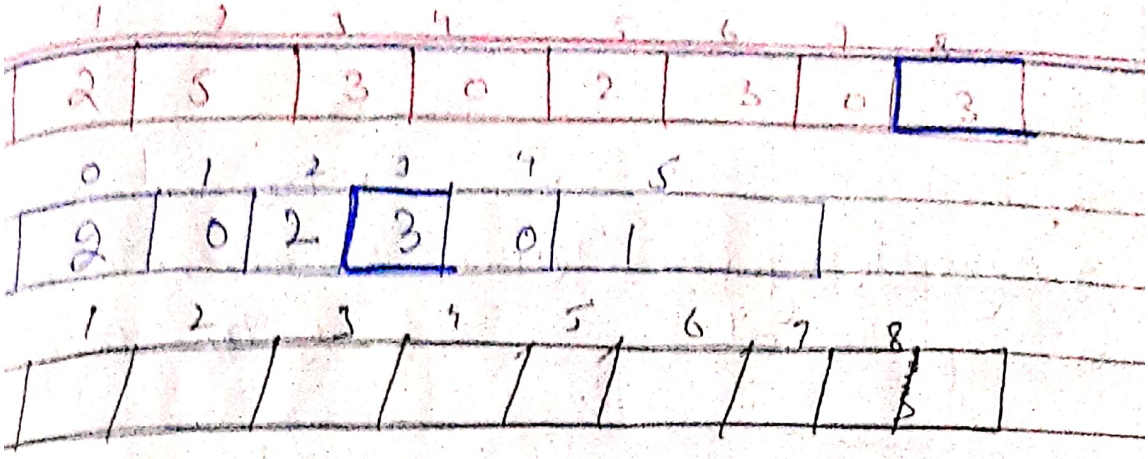
Executing loop 1:



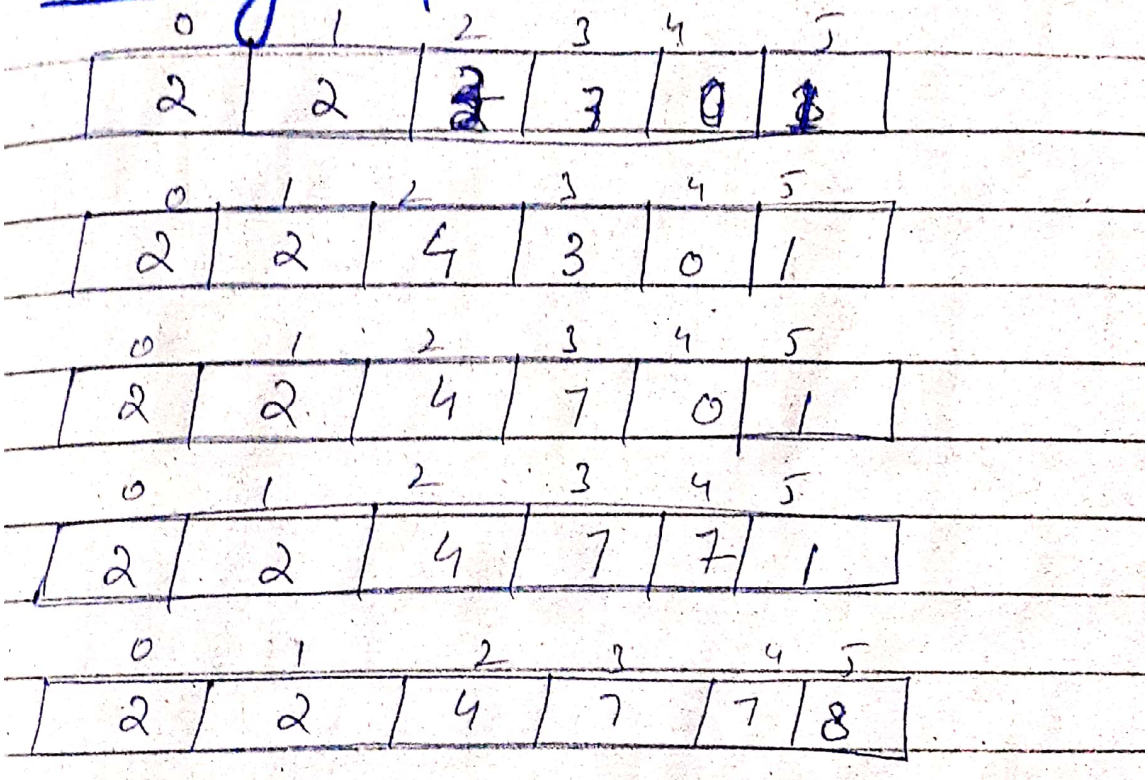
Executing loop 2:




A: 2 5 3 0 2 3 0 3
C: 1 0 1 1 0 1
B:
A: 2 5 3 0 2 3 0 3
C: 1 0 2 1 0 1
B:
A: 2 5 3 0 2 3 0 3
C: 1 0 2 2 0 1
B:
A: 2 5 3 0 2 3 0 3
C: 2 0 2 2 0 1
B:

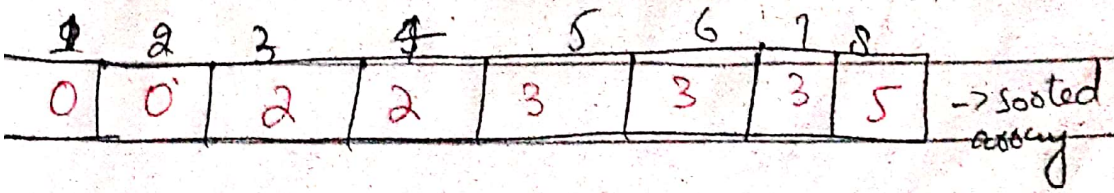


Executing loop -3:



Executing Loop 4:

Final result will be:



Pros:

- ☆ Fast
- ☆ Asymptotically fast - O(n+k)
- ☆ Simple to code

Con's:

- ☆ Does not sort in
- ☆ Requires O(n+
  extra storage

Radix Sort

=> Radix sort is non-comparative sorting algorithm.

=> Two classifications of radix sorts are least significant radix sorts and most significant radix sorts.

=> LSB radix sorts process the integer representations starting from the least significant and move towards the most significant digits

=> Assumption:

Input taken from large set of numbers

=> Requires only d passes through the list

=> The input data must be betweeen a range of elements.

Stable: Yes

in-place: No

Time complexity: O(d(n+K))

Note: How many times counting sort will be apply?

=> no of digits time.

Algorithm:-

```
Radixsort(A, d)
    for i = 1 to d ——— Θ(d)
        use a stable sort to sort array ——— (n+k)
        A on digit i
```

Routine / function:

```
Radix_sort(A, n)
  {
  int  max = getmax(A, n)
for( pos = 1 ; max/pos > 0 , pos*10)
    {
        Countsort(A, n, pos)
    }
}
Countsort(int A[], int n, int pos)
  {
      int count[ 10] = {0};
    for (i = 0 ; i < n ; i++)
    ++ count [ (a[i]/pos) %10];
    for (i = 1, i <= k ; i++)
        Count[i] = count[i] + count[i-1];
```

```
for(i=n-1 ; i>=0 ; i--)
    B[Count[(a[i]/Pw)%10]] = a[i]
--count[(a[i]/Pw)%10]
for i=1 to A.length
    A[i]=B[i]
}
```

Example:

189	986	205	421	097	192	535

Input	1st Pass	Pass 2	Pass 3
189	421	205	097
986	192	421	189
205	205	535	192
421	535	189	205
097	986	986	421
192	097	097	535
535	189	192	986

Buket Sort:

=> it is used when input is uniformly distributed over a range [0-1]

=> Works on floating point number between range 0.0 to 1.0.

In-place:
Not

Stable: Yes or Not be

Time Complexity:
=> $O(1) \times n$
$\in O(n)$

Note:
(Use scatter gather approach)

Algorithm:-

```
BucketSort(A)
    Let B[0..n-1] be a new array
    n = A.length
    for i=0 to n-1 ——— n
        make B[i] an empty list ——
    for i = 1 to n ——— n
        insert A[i] into list B[⌊n A[i]⌋] ——— n
    for i=0 to n-1 ———
        Sort list B[i] with insertion
    Concatenate the lists together
    B[0], B[1], ... B[n-1] (in order) ——
```

Example:

	A
1	.78
2	.17
3	.39
4	.26
5	.72
6	.94
7	.21
8	.12
9	.23
10	.68

B

0 → /

1 → .12 → .17 → /

2 → .21 → .23 → .26 → /

3 → .39 → /

4 → /

5 → /

6 → .68 → /

7 → .72 → .78 → /

8 → /

9 → .94 → /

10

1	2	3	4	5	6	7	8	9	10
.12	.17	.21	.23	.26	.39	.68	.72	.78	.94